

BỘ KHOA HỌC VÀ CÔNG NGHỆ

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG



BÁO CÁO THỰC TẬP CƠ SỞ

**Đề tài: Tìm hiểu và ứng dụng mạng nơ-ron tích
chập (CNN) trong bài toán phân loại hình ảnh
giống cây bệnh.**

Giảng viên hướng dẫn	: ThS. Bùi Văn Kiên
Họ và tên sinh viên	: Khổng Thị Thuỳ Linh
Mã sinh viên	: B23DCCN483
Lớp	: D23CQCN07-B
Nhóm	: 25

Hà Nội – 2026

Mục lục

<u>I.</u>	<u>PHẦN LÝ THUYẾT</u>	<u>3</u>
1.1.	GIỚI THIỆU DEEP LEARNING.....	3
1.2.	MẠNG NƠ-RON NHÂN TẠO.....	5
1.3.	CẤU TRÚC CNN	9
1.3.1.	CONVOLUTIONAL LAYER.....	9
1.3.2.	POOLING LAYER	10
1.3.3.	FULLY CONNECTED LAYER.....	12
<u>II.</u>	<u>PHẦN THỰC NGHIỆM</u>	<u>14</u>
2.1.	THU THẬP DATASET ẢNH.....	14
2.1.3.	XÂY DỰNG MÔ HÌNH CNN BẰNG PYTHON + TENSORFLOW/KERAS VỚI BÀI TOÁN PHÂN LOẠI GIỐNG CÂY BỆNH.....	17
2.2.	ĐÁNH GIÁ.....	22
2.2.1.	ACCURACY	22
2.2.2.	LOSS	23
2.2.3.	SỐ SÁNH VỚI MODEL CÓ SẴN (MOBILENET)	24
<u>III.</u>	<u>TÀI LIỆU THAM KHẢO</u>	<u>31</u>

I. Phần lý thuyết

1.1. Giới thiệu Deep Learning

a. Quá trình hình thành

Trước khi tìm hiểu về quá trình hình thành của Deep Learning, ta cần nắm rõ các thuật ngữ quan trọng: *Artificial Intelligence (AI)* và *Machine Learning (ML)*

* *Artificial Intelligence (AI)*: hay còn được gọi là trí tuệ nhân tạo, được định nghĩa là **bản sao của trí tuệ con người** trong máy tính.

* *Machine Learning (ML)*: hay còn được gọi là học máy, ML hướng đến **khả năng của một cái máy để học cách sử dụng lượng lớn tập dữ liệu (dataset)**, thay vì các quy tắc đã được đặt sẵn cho nó.

Vd: Trong lập trình truyền thống, con người sẽ viết luật sẵn cho máy để máy xử lý:

- Máy chỉ nhận số chẵn: Nếu số chia hết cho 2 → số chẵn → nhận. Ở đây, con người nghĩ luật và máy chỉ việc làm theo.

Nhưng Machine Learning thì khác, với ML ta không đưa luật chi tiết, mà đưa cho máy rất nhiều dữ liệu (dataset) kèm theo với đáp án đúng. Khi đó, máy sẽ tự suy ra quy luật và tự học cách đưa ra dự đoán. Cách học này lợi dụng sức mạnh xử lý của máy tính hiện đại, những chiếc máy có thể xử lý được lượng lớn dữ liệu.

Supervised Learning và Unsupervised Learning.

* *Supervised Learning (Học có giám sát)*: **bao gồm việc sử dụng các tập dữ liệu có dán nhãn (labelled dataset) mà có các đầu vào và đầu ra được dự đoán.** Nói cách khác, là học bằng dữ liệu có sẵn đáp án đúng.

Vd:

Ảnh (Input)	Label
Ảnh con mèo	“Mèo”
Ảnh con cún	“Cún”

Máy học rất nhiều cặp I/O như vậy, sau khi học xong, gặp ảnh mới chưa từng học → nó tự đoán.

* *Unsupervised Learning (Học không có giám sát)*: Là **nhiệm vụ của học máy để sử dụng các tập dữ liệu không có cấu trúc cụ thể.** Khi đó, ta đưa cho máy tập dữ liệu nhưng không có đáp án đúng (không có label), máy phải tự tìm cấu trúc, nhận ra quy luật và phân nhóm dữ liệu.

Vd: Khi người dùng đưa cho máy 10.000 ảnh đủ loại con vật, nhưng không cho đáp án. Máy sẽ tự phân loại những nhóm ảnh có đặc điểm giống nhau, điểm chung.

→ Thay vì “Đây là mèo”, máy sẽ chạy: “Mấy bức ảnh này trông giống nhau, gom một nhóm”

b. Cách Deep Learning hoạt động

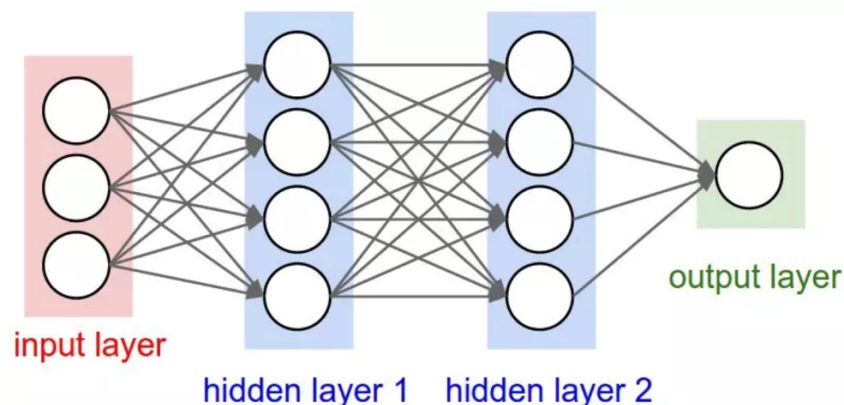
**Deep learning là gì?*

Deep Learning là một nhánh của Machine Learning dùng mạng nơ-ron nhân tạo nhiều lớp. (deep = nhiều tầng xử lý). Nó cho phép chúng ta huấn luyện một AI có thể dự đoán được các output dựa trên một tập các đầu vào. Cả hai phương pháp có giám sát và không giám sát đều có thể sử dụng để huấn luyện.

Nó được lấy cảm hứng từ não của con người trong việc xử lý thông tin.

**Deep learning hoạt động như thế nào?*

Do lấy cảm hứng từ bộ não của con người, bộ não của AI cũng có các nơ-ron. Deep learning hoạt động qua các lớp (layer) trong mạng nơ-ron.



**Nguồn ảnh: <https://viblo.asia/p/gioi-thieu-ve-deep-learning-deep-learning-hoat-dong-nhu-the-nao-3Q75w2nDlWb>*

Như trên hình ảnh minh họa, bên trong mỗi lớp là rất nhiều nơ-ron, chúng được biểu diễn bằng các vòng tròn.

1. Input layer

Đây là lớp nhận dữ liệu đầu vào, với CNN, đầu vào là ảnh (pixel).

2. Các Hidden layer

Các lớp ẩn sẽ xử lý dần dần, mỗi lớp sẽ học một đặc trưng khác nhau.

Vd:

Lớp (layer)	Mục tiêu học
Layer 1	Đường thẳng, cạnh màu
Layer 2	Hình dạng, hoa văn

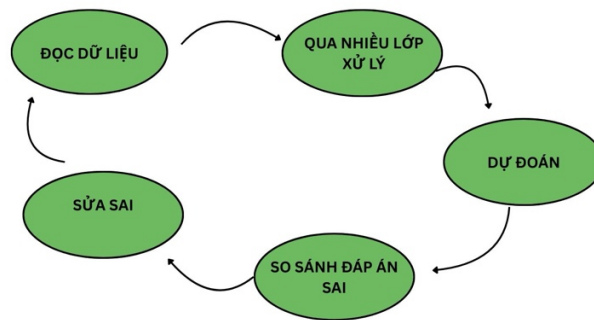
Layer 3	Bộ phận vật thể (mắt, tai, chân)
Layer 4	Nhận diện đối tượng hoàn chỉnh

Càng vào lớp sâu → Càng hiểu đối tượng rõ hơn.

3. Output layer

Nếu là bài toán có giám sát, sau khi đưa ra output layer mô hình sẽ so sánh với đáp án đúng và tính toán sai số. Cuối cùng là bước sử dụng thuật toán để điều chỉnh lại các trọng số để lần sau đoán chính xác hơn.

Tổng kết: Deep Learning học theo mô hình sau:



1.2. Mạng nơ-ron nhân tạo

Vậy một nơ-ron nhân tạo thực sự làm gì để máy móc có thể tính toán được kết quả để truyền sang lớp sâu hơn? Để tìm hiểu điều đó, trước hết chúng ta cần hiểu rõ về khái niệm Mạng nơ-ron nhân tạo.

1.2.1. Khái niệm mạng nơ-ron nhân tạo

Mạng nơ-ron nhân tạo (Neural Network) là một mô hình học máy được thiết kế theo dạng mô phỏng các tế bào thần kinh bên trong não bộ con người trong việc truyền tín hiệu và xử lý thông tin.

1.2.2. Cấu trúc cơ bản của mạng nơ-ron nhân tạo

Cấu trúc của mạng nơ-ron nhân tạo gồm có 3 lớp cơ bản: *Input Layer*, *Hidden Layer* và *Output Layer*. Cấu trúc này khá giống với cấu trúc của Deep Learning, vì Deep Learning không phải mô hình khác hoàn toàn, mà là phiên bản mạng nơ-ron nhân tạo nâng cấp hơn, có nhiều lớp ẩn hơn.

- **Input layer:** Đây là lớp nằm bên trái của mạng, nơi dữ liệu đi vào mạng nơ-ron.
- **Hidden layer:** Nằm giữa input layer và output layer, nơi mạng nơ-ron thực sự học và xử lý thông tin. → Đây là lúc mạng học các mẫu.

- **Output layer:** Đây là lớp nằm phía bên phải và là lớp cuối cùng của mạng nơ-ron, nơi mô hình đưa ra kết quả dự đoán cuối cùng dựa trên quá trình xử lý các lớp trước đó.

Ngoài ra, để tạo thành một mạng nơ-ron nhân tạo còn cần có các thành phần cơ bản sau:

- **Tế bào thần kinh (Neurons):** Đây là các đơn vị tính toán cơ bản trong mạng nơ-ron, tương tự như các tế bào thần kinh trong não người. Mỗi nơ-ron nhận tín hiệu đầu vào (các giá trị số), áp dụng trọng số và độ lệch, sau đó sử dụng một hàm kích hoạt để tính toán đầu ra. Kết quả này được truyền đến các nơ-ron khác trong mạng để tiếp tục xử lý.
- **Kết nối (Connections):** Các kết nối giữa các nơ-ron là các liên kết truyền tải thông tin từ nơ-ron này sang nơ-ron khác. Mỗi kết nối có một giá trị trọng số (weight) để xác định mức độ ảnh hưởng của nó đối với đầu ra của nơ-ron nhận thông tin.
- **Trọng số và Độ lệch (Weights and Biases):** Trọng số là tham số điều chỉnh độ mạnh yếu của các kết nối giữa các nơ-ron, quyết định mức độ ảnh hưởng của một đầu vào đến kết quả tính toán của nơ-ron. Còn độ lệch (bias) giúp điều chỉnh các đầu vào sao cho mạng nơ-ron có thể học chính xác hơn, đặc biệt khi đầu vào không có giá trị rõ ràng.
- **Truyền tiến và truyền ngược (Forward & Backward Propagation):** Đây là các cơ chế giúp truyền tải và xử lý dữ liệu qua các lớp nơ-ron.
- **Quy tắc học (Learning Rule):** Để mạng nơ-ron học được từ dữ liệu, quy tắc học đóng vai trò giúp mạng điều chỉnh trọng số và độ lệch qua thời gian, từ đó cải thiện độ chính xác của mạng.

1.2.3. Nguyên lý hoạt động của mạng nơ-ron nhân tạo

**Forward Propagation (Truyền tiến)*

Khi dữ liệu được đưa vào mạng, nó sẽ di chuyển qua các lớp của mạng nơ-ron từ lớp đầu vào đến lớp đầu ra. Quá trình này được gọi là truyền dữ liệu theo chiều tiến (forward propagation). Trong giai đoạn này, mạng nơ-ron nhân tạo hoạt động như sau:

- **Biến đổi tuyến tính:** Mỗi nơ-ron trong một lớp nhận đầu vào, sau đó nhân các giá trị này với các trọng số (weight) của các kết nối. Các giá trị này được cộng lại với nhau và một độ lệch (bias) được thêm vào. Cách tính này có thể được diễn tả dưới dạng toán học như sau:

$$z = w_1 * x_1 + w_2 * x_2 + \dots + w_n * x_n + w_0$$

(Trong đó w là trọng số, z là đầu vào và b là độ lệch)

Hàm kích hoạt: Kết quả của phép biến đổi tuyến tính (ký hiệu là z) sau đó được đưa qua một hàm kích hoạt. Hàm kích hoạt có vai trò quan trọng trong việc giới thiệu tính phi tuyến vào hệ thống giúp mạng nơ-ron học và nhận diện các mẫu phức tạp hơn. Một số hàm kích hoạt phổ biến gồm ReLU, sigmoid, tanh.

***Backward propagation (Truyền ngược)**

Sau khi quá trình truyền tiến hoàn tất, mạng sẽ đánh giá độ chính xác của kết quả bằng một hàm mất mát (loss function). Hàm mất mát sẽ đo lường sự chênh lệch giữa kết quả dự đoán của mạng và kết quả thực tế.

Quá trình truyền ngược giúp tối ưu hoá mạng nơ-ron thông qua các bước:

- Tính toán hàm mất mát: Sau khi forward propagation (truyền tiến) xong, mạng có kết quả dự đoán, hàm mất mát sẽ trả lời cho câu hỏi: **kết quả đã sai bao nhiêu?**
 - Sai ít $\rightarrow$ loss nhỏ
 - Sai nhiều $\rightarrow$ loss lớn
 - VD: Bài toán hồi quy $\rightarrow$ Dùng MSE, bài toán phân loại $\rightarrow$ dùng Cross-Entropy.
- Tính toán Gradient (độ dốc): Sau khi biết sai bao nhiêu, mạng cần biết: **Phải chỉnh trọng số theo hướng nào để giảm lỗi?** Gradient chính là: hướng + mức độ thay đổi của loss khi thay đổi từng tham số. Mạng dùng quy tắc chuỗi (chain rule) để:
 - Lan truyền lỗi từ lớp đầu ra
 - Ngược về các lớp trước
 - Vì vậy gọi là backpropagation (truyền ngược)
 - Hiểu đơn giản, tính Gradient chính là tìm xem mỗi trọng số góp phần gây lỗi bao nhiêu, từ đó tìm ra hướng chỉnh
- Cập nhật trọng số: Sau khi biết hướng chỉnh, mạng điều chỉnh trọng số để giảm lỗi
 - Công thức ý tưởng: trọng số mới = trọng số cũ - learning rate * gradient
 - Gradient: hướng cần đi
 - Learning rate: bước đi dài hay ngắn

***Lặp lại quá trình (Iteration)**

Quá trình truyền dữ liệu theo chiều tiến, tính toán hàm mất mát, truyền ngược dữ liệu và cập nhật trọng số sẽ được lặp đi lặp lại qua nhiều vòng (iterations) trên toàn bộ tập dữ liệu.

Qua mỗi vòng lặp, hàm mất mát sẽ giảm dần và dự đoán của mạng nơ-ron ngày càng chính xác hơn.

1.2.4. Các phương pháp học của mạng nơ-ron nhân tạo

- Học có giám sát (Supervised Learning)
- Học không giám sát (Unsupervised Learning)

- Học tăng cường (Reinforcement Learning): Học tăng cường là phương pháp mà mạng nơ-ron học bằng cách thử và sai, giống như cách con người và động vật học từ trải nghiệm
 - VD: Hãy tưởng tượng khi bạn đang chơi một trò chơi
 - Khi bạn làm đúng → bạn được điểm thưởng (reward)
 - Khi bạn làm sai → bạn bị phạt (penalty)
 - Mạng sẽ cố gắng tìm ra chiến lược tối ưu giúp tối đa hoá tổng phần thưởng trong suốt quá trình học.

1.2.5. Ứng dụng của mạng nơ-ron nhân tạo đối với đề tài bài toán phân loại hình ảnh

1. Nhận diện khuôn mặt
 - Mở khoá điện thoại bằng khuôn mặt
 - Xác thực danh tính trong hệ thống bảo mật
 - Gắn thẻ người trong ảnh
 - VD: Hệ thống face ID, camera an ninh
2. Chẩn đoán y khoa từ ảnh
 - Phân loại ảnh X-quang (có bệnh/ không bệnh)
 - Phát hiện khối u trong ảnh MRI hoặc CT
 - Giúp bác sĩ hỗ trợ chẩn đoán nhanh và chính xác hơn
3. Xe tự lái
 - Nhận diện biển báo giao thông
 - Phân loại vật thể: người đi bộ, xe cộ, làn đường
 - Là một thành phần quan trọng trong hệ thống lái tự động
4. Kiểm tra chất lượng sản phẩm
 - Phát hiện lỗi trên dây chuyền sản xuất
 - Phân loại sản phẩm đạt/ không đạt
 - Ứng dụng nhiều trong sản xuất công nghiệp
5. Tìm kiếm và gợi ý hình ảnh
 - Tìm ảnh tương tự trong thư viện
 - Gợi ý sản phẩm theo hình ảnh
 - Dùng trong thương mại điện tử và mạng xã hội
6. Phân loại nội dung
 - Phát hiện nội dung không phù hợp
 - Lọc ảnh spam hoặc ảnh nhạy cảm
 - Giúp kiểm duyệt nội dung tự động

1.2.6. Thách thức và hạn chế của mạng nơ-ron nhân tạo.

- **Thiếu quy tắc cấu trúc mạng:** Không có quy tắc cụ thể để xác định cấu trúc mạng, việc chọn kiến trúc phù hợp phụ thuộc vào thử nghiệm và kinh nghiệm
- **Tốn kém tài nguyên tính toán:** Huấn luyện mạng nơ-ron yêu cầu tài nguyên tính toán lớn, điều này có thể trở thành rào cản đối với các tổ chức có nguồn lực hạn chế.

- **Phụ thuộc vào phần cứng:** Mạng nơ-ron cần bộ xử lý có khả năng xử lý song song, khiến việc triển khai phụ thuộc vào phần cứng đặc biệt như GPU
- **Chuyển đổi dữ liệu thành số:** Mạng nơ-ron chỉ làm việc với dữ liệu dạng số, yêu cầu quá trình chuyển đổi trước khi xử lý.
- **Kết quả không chính xác:** Nếu huấn luyện không đúng, mạng nơ-ron có thể đưa ra kết quả không chính xác hoặc thiếu sót .
- **Hiện tượng overfitting:** Mạng nơ-ron dễ bị overfitting khi huấn luyện với dữ liệu nhỏ, làm giảm khả năng tổng quát hoá khi áp dụng cho dữ liệu mới.

1.3. Cấu trúc CNN

Convolution Neural Network (CNN) là một loại mô hình trong lĩnh vực Deep Learning được thiết kế đặc biệt để xử lý ảnh. Nói một cách đơn giản, CNN là một mạng nơ-ron có khả năng “nhìn” và hiểu hình ảnh.

Để chỉ ra điểm đặc biệt của CNN, trước hết ta cùng lục lại về các phương pháp cũ: để máy tính nhận diện hình ảnh, con người phải:

- Tự viết thuật toán để tìm cạnh
- Tìm góc
- Tìm màu sắc
- Tìm hình dạng

→ Việc này gọi là trích xuất đặc trưng thủ công.

CNN làm theo một cách hoàn toàn khác, CNN tự học đặc trưng từ dữ liệu. Ví dụ với bài toán phân loại giống cây bệnh, khi huấn luyện CNN với hình ảnh lá cây

- Các lớp đầu sẽ học: cạnh lá, đường gân lá
- Các lớp sau sẽ học hình dạng lá, đốm bệnh
- Các lớp sâu hơn sẽ hiểu: loại bệnh của cây

Nói về cách hoạt động của CNN, CNN sử dụng phép toán gọi là:

Convolution (tích chập).

Hiểu một cách đơn giản về Convolution:

- Một bộ lọc nhỏ (filter/kernel) sẽ quét qua ảnh
- Bộ lọc này giúp phát hiện đặc trưng như: cạnh, góc, texture

Để tìm hiểu về CNN, trước hết chúng ta tìm hiểu về các lớp cơ bản và nguyên lý hoạt động của Convolutional Neural Networks.

1.3.1. Convolutional Layer

Convolutional layer là lớp quan trọng nhất trong CNN. Nhiệm vụ của lớp này là: trích xuất đặc trưng (features) từ ảnh

Ví dụ với ảnh chiếc lá, CNN có thể phát hiện:

- Cạnh của lá
- Đường gân
- Đốm bệnh
- Hình dạng lá

Lớp Convolution này sử dụng một bộ lọc (kernel) để quét qua từng vùng nhỏ của hình ảnh và thực hiện phép nhân tích chập (convolution) giữa các giá trị pixel với trọng số của bộ lọc.

Kernel là một ma trận nhỏ, ví dụ về kernel 3x3:

$$\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

Kernel sẽ quét qua toàn bộ ảnh, mỗi lần quét:

- Lấy một vùng nhỏ của ảnh
- Nhân từng pixel với giá trị trong kernel
- Cộng lại

→ Phép toán này gọi là Convolution(tích chập)

Kết quả của quá trình này tạo thành **bản đồ đặc trưng (feature map)**, giúp mô hình phát hiện các đặc điểm như cạnh, góc, màu sắc hoặc kết cấu trong ảnh.

Feature map cho biết:

- Vị trí cạnh
- Vị trí góc
- Vị trí kết cấu

Ví dụ: Khi quét qua ảnh góc, Feature map chỉ giữ lại những đặc trưng quan trọng.

CNN thường dùng nhiều kernel khác nhau, nên sẽ tạo ra nhiều feature map.

Các tham số quan trọng của lớp tích chập bao gồm: Số lượng bộ lọc, Stride (bước di chuyển của bộ lọc) và Padding (giữ kích thước ảnh). Trong đó:

- Stride là bước di chuyển của kernel.
Ví dụ: Stride = 1, kernel di chuyển từng pixel một
Stride = 2, kernel nhảy 2 pixel
→ Stride lớn hơn thì feature map sẽ nhỏ hơn.
- Padding là việc thêm pixel vào viền ảnh. Mục đích là giữ kích thước ảnh và tránh mất thông tin ở viền ảnh.

Sau mỗi phép tích chập, Convolutional Neural Networks thường áp dụng hàm kích hoạt ReLU (Rectified Linear Unit) để loại bỏ giá trị âm, tăng tính phi tuyến và giúp mô hình học hiệu quả hơn.

Công thức : $f(x) = \max(0, x)$

Tổng kết: Convolutional layer chịu trách nhiệm trích xuất các đặc trưng từ dữ liệu đầu vào.

1.3.2. Pooling layer

Sau khi ảnh đi qua Convolutional Layer, ta thu được feature map. Lúc này kích thước feature map vẫn còn khá lớn, nên Pooling layer được sử dụng để:

- Giảm kích thước feature map
- Giảm số lượng tham số
- Tăng tốc độ tính toán

- Giảm nguy cơ overfitting

Trong CNN, pooling giống như bước nén thông tin quan trọng của ảnh.

**Cách hoạt động của pooling:*

Pooling sử dụng một cửa sổ nhỏ (2x2 hoặc 3x3) để quét qua feature map.

Thay vì giữ lại tất cả giá trị pixel, pooling chỉ giữ lại một giá trị đại diện cho vùng đó.

Ví dụ: có ma trận ban đầu (feature map) :

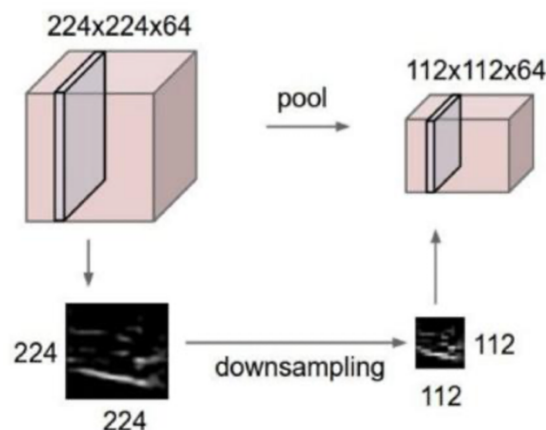
$$\begin{bmatrix} 4 & 2 & 1 & 3 \\ 6 & 5 & 2 & 1 \\ 7 & 3 & 4 & 2 \\ 1 & 2 & 3 & 4 \end{bmatrix}$$

Ta dùng Max Pooling 2x2, chia ma trận thành các vùng 2x2, ta có kết quả sau pooling:

$$\begin{bmatrix} 6 & 3 \\ 7 & 4 \end{bmatrix}$$

**Ta có 2 loại pooling phổ biến*

- Max Pooling: Như ví dụ ở trên, cách hoạt động của Max Pooling là chọn giá trị lớn nhất trong vùng quét. Từ đó, Max Pooling giúp **giữ lại đặc trưng mạnh nhất** và **giúp mô hình tập trung vào chi tiết nổi bật**.
→ Vì vậy Max Pooling được dùng nhiều nhất trong CNN.
- Average Pooling: Cách hoạt động là lấy giá trị trung bình của vùng quét. Từ đó **giúp tổng hợp thông tin** thay vì chỉ giữ giá trị lớn nhất như Max Pooling



**Pooling giúp tránh Overfitting*

Overfitting là hiện tượng:

- Mô hình học hoá quá kỹ dữ liệu huấn luyện
- Tuy nhiên, dự đoán kém với dữ liệu mới

Pooling giúp giảm overfitting vì:

- Giảm số lượng dữ liệu cần học
- Làm mô hình đơn giản hơn
- Giúp mô hình tổng quát tốt hơn.

1.3.3. Fully Connected Layer

Fully Connected Layer là lớp cuối cùng trong CNN với nhiệm vụ: dùng các đặc trưng đã trích xuất để đưa ra kết quả phân loại cuối cùng.

Các lớp trước (Convolution + Pooling) chỉ làm việc: tìm cạnh, tìm hình dạng, tìm đặc trưng của ảnh, Fully Connected Layer mới là lớp quyết định ảnh thuộc loại nào (phân loại ảnh)

**Fully Connected (Kết nối đầy đủ)*

Trong lớp này, mỗi nơ-ron sẽ kết nối với tất cả nơ-ron ở lớp trước

Ví dụ: Lớp trước có 100 nơ-ron → một nơ-ron ở Fully Connected sẽ nhận 100 giá trị đầu vào. Do đó gọi là “kết nối đầy đủ”

**Flattening (làm phẳng)*

Trước khi vào Fully Connected Layer, dữ liệu thường là feature map dạng ma trận, CNN sẽ biến nó thành một vector một chiều. Quá trình này gọi là Flattening.

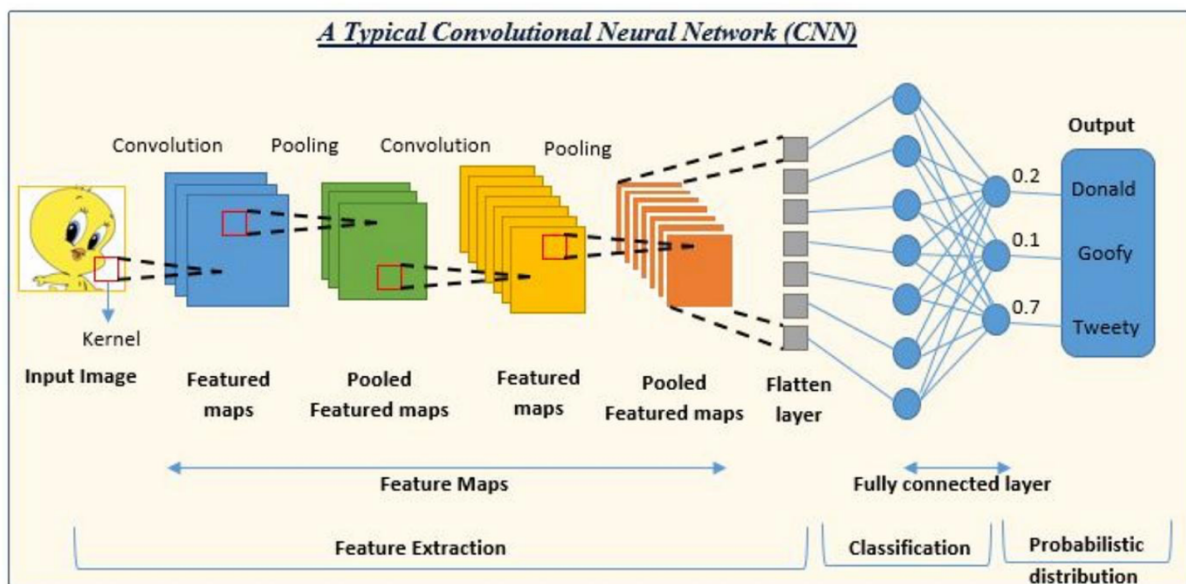
Ví dụ: $\begin{bmatrix} 2 & 4 \\ 3 & 5 \end{bmatrix} \rightarrow [2, 4, 3, 5]$

Ma trận → chuỗi số một chiều để đưa vào mạng nơ-ron.

**Hàm kích hoạt ở lớp cuối*

Sau khi xử lý ở Fully Connected Layer, CNN sử dụng hàm kích hoạt để tính xác suất cho từng lớp. Hai hàm phổ biến

- Softmax: thường được dùng khi phân loại nhiều lớp (VD: chó, mèo, ô tô)
- Sigmoid: thường dùng khi phân loại 2 lớp (bệnh / không bệnh) hoặc multi-label classification



Hình 1. Why Matrix Multiplication Matters in Deep Learning | by Farid khan | Medium

1.3.4. Additional layer

Ngoài các lớp chính của CNN như Convolutional Layer, Pooling Layer, Fully Connected Layer, CNN còn có một số lớp phụ giúp:

- Mô hình học tốt hơn
- Huấn luyện nhanh hơn
- Giảm lỗi khi dự đoán

**Activation Layer (lớp kích hoạt):*

Activation Layer là lớp áp dụng hàm kích hoạt phi tuyến cho các giá trị đầu vào. Với mục đích:

- Giúp mô hình học các mối quan hệ phức tạp
- Nếu không có activation, mạng chỉ giống như một phép toán tuyến tính đơn giản
- Các hàm kích hoạt phổ biến:
 - ReLU: $f(x) = \max(0, x)$
Ưu điểm: Tính toán nhanh và hiệu quả nên được dùng nhiều nhất trong CNN
 - Sigmoid: $f(x) = 1/(1 + e^{-x})$. Kết quả nằm trong khoảng $0 \rightarrow 1$
Thường dùng cho bài toán phân loại nhị phân
 - Tanh (Hyperbolic Tangent): giá trị nằm trong khoảng $-1 \rightarrow 1$.
Tanh giúp dữ liệu cân bằng quanh 0.

**Dropout Layer*

Dropout Layer là kỹ thuật giúp giảm overfitting

Cách hoạt động: Trong quá trình huấn luyện: Dropout sẽ tắt ngẫu nhiên một số nơ-ron.

→ Kết quả:

- Mô hình không phụ thuộc vào một số nơ-ron cụ thể
- Khả năng tổng quát hoá tốt hơn

**Batch Normalization Layer*

Batch Normalization là lớp chuẩn hoá dữ liệu trong mạng, giúp dữ liệu ổn định hơn và giúp mạng học nhanh hơn

Cách hoạt động: Batch Normalization:

- Lấy một batch dữ liệu
- Tính trung bình và độ lệch chuẩn
- Chuẩn hoá dữ liệu

Tác dụng:

- Giảm hiện tượng Gradient bị quá lớn hoặc quá nhỏ
- Tăng tốc quá trình huấn luyện

1.3.5. Ba thành phần quan trọng trong cấu trúc CNN

Ngoài các lớp, CNN còn có 3 cơ chế quan trọng giúp mô hình hoạt động hiệu quả:

**Local Receptive Field (Trường tiếp nhận cục bộ)*

Trong mạng nơ-ron truyền thống, **mỗi nơ-ron kết nối với tất cả pixel của ảnh**. Điều này làm: số tham số rất lớn + tính toán chậm.
Để giải quyết những khuyết điểm này, CNN chỉ nhìn một vùng nhỏ của ảnh.
Ví dụ: Với ảnh 32x32 pixel, kernel 3x3, một nơ-ron chỉ xử lý 3x3 pixel chứ không phải toàn bộ ảnh

Như vậy, với cơ chế Local Receptive Field, CNN giúp:

- Phát hiện đặc trưng theo từng khu vực nhỏ
- Giảm số tham số
- Tăng hiệu quả tính toán.

***Shared Weights (Trọng số chia sẻ)**

Trong CNN, một kernel được dùng cho toàn bộ ảnh.

Ví dụ:

Kernel phát hiện cạnh ngang:

$$\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

Kernel này sẽ: quét toàn bộ ảnh và phát hiện cạnh ở mọi vị trí

Như vậy, với cơ chế Shared Weights (Trọng số chia sẻ), CNN giúp :

- Giảm số lượng tham số
- Giúp mô hình nhận diện cùng một đặc trưng ở nhiều vị trí khác nhau

***Pooling layer**

Pooling có nhiệm vụ:

- Giảm kích thước feature map
- Giữ lại thông tin quan trọng

Có hai loại pooling phổ biến: Max pooling và Average pooling (đã giải thích rõ ở phần 1.3.2)

Qua 3 cơ chế kể trên, ta có thể suy ra: CNN học đặc trưng theo nhiều cấp độ.
Nói một cách cụ thể hơn, điểm mạnh của CNN là học **đặc trưng từ đơn giản đến phức tạp**.

II. Phần thực nghiệm

2.1. Thu thập dataset ảnh

2.1.1. Vai trò của dataset ảnh trong bài toán phân biệt cây bệnh

1. Cung cấp thông tin để mô hình học

Dataset ảnh cung cấp các hình ảnh lá cây, thân cây hoặc quả bị bệnh và không bị bệnh.

Mô hình CNN sẽ học từ các ảnh này để:

- Nhận diện đặc trưng bệnh như: đốm lá, đổi màu, vết cháy, nấm mốc
- Phân biệt giữa các loại bệnh khác nhau hoặc cây khỏe mạnh và cây bị bệnh

→ Nói đơn giản, dataset chính là nguồn kiến thức để mô hình học.

2. Giúp mô hình nhận diện đặc trưng bệnh

Các mô hình CNN sẽ sử dụng dataset ảnh để học các đặc trưng trực quan (visual features) như:

- Màu sắc của lá
- Kết cấu của bề mặt lá
- Hình dạng của vết bệnh
- Vị trí và kích thước của các đốm bệnh

→ Nhờ dataset, CNN có thể tự động trích xuất đặc trưng thay vì phải thiết kế thủ công.

3. Đánh giá độ chính xác của mô hình

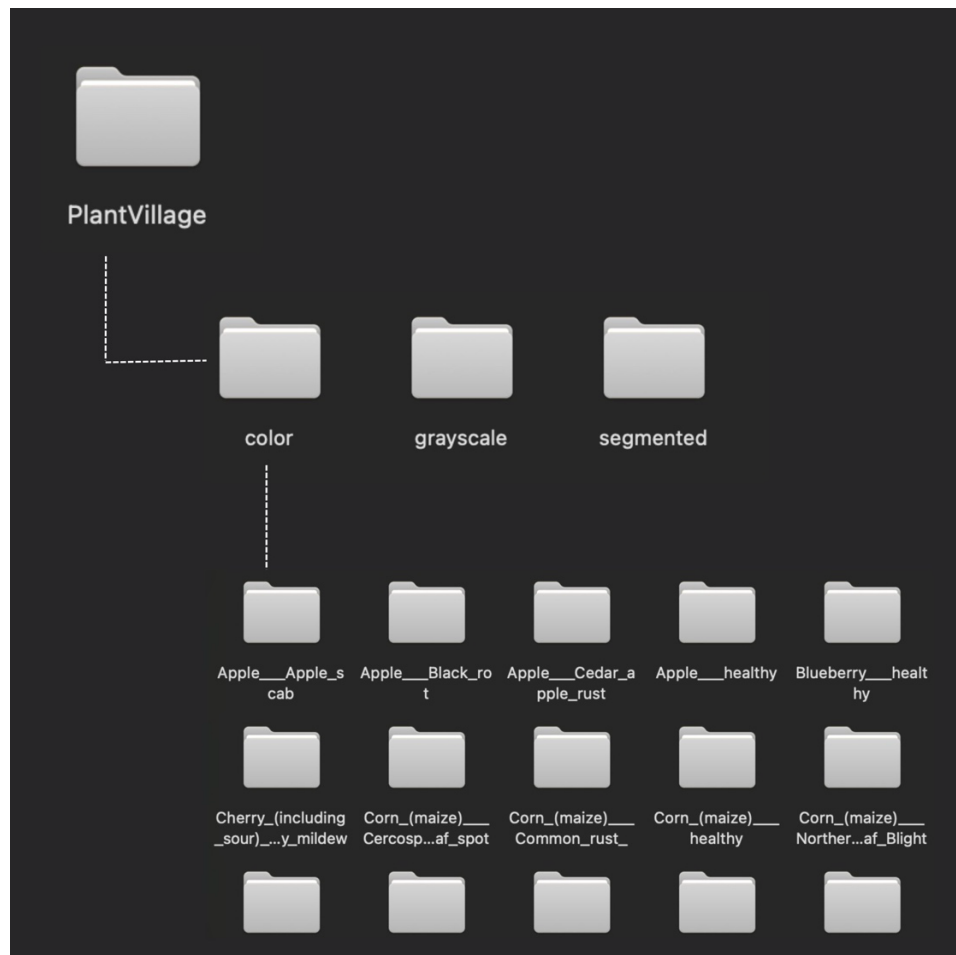
Dataset thường được chia thành các phần:

- Training set: dùng để huấn luyện mô hình
- Validation set: điều chỉnh tham số
- Test set: đánh giá mô hình

2.1.2. Thu thập dataset ảnh

1. Tải dataset

Trong bài báo cáo này, em dùng dataset tải từ nguồn công khai: Kaggle Dataset PlantVillage chứa hơn 50000 ảnh của nhiều loại cây và bệnh khác nhau



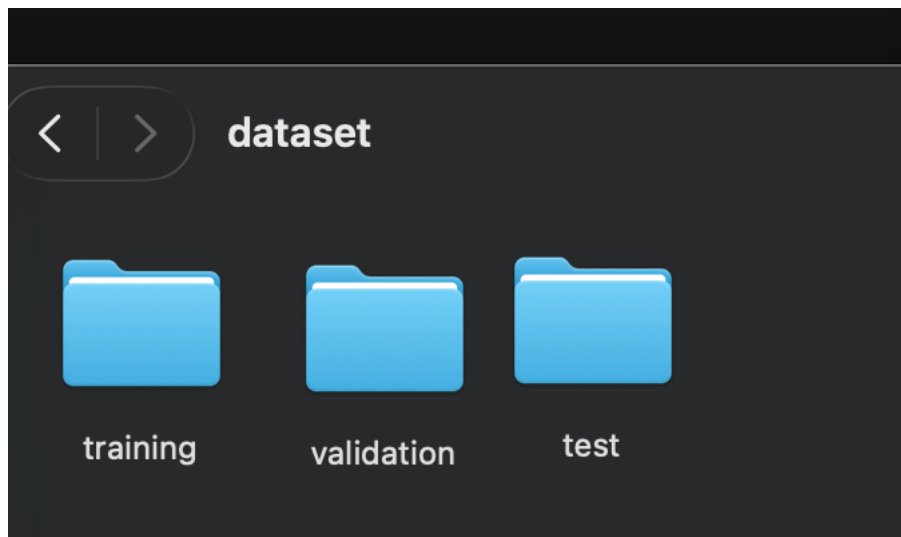
2. Gắn nhãn dữ liệu

Đây là một bước quan trọng vì CNN học dựa vào nhãn. Nếu như sai nhãn thì mô hình sẽ học sai hoàn toàn.

```
Dataset/  
  Healthy/  
  Leaf_blight/  
  Powdery_mildew/
```

3. Chuẩn hoá dữ liệu trước khi đưa vào CNN

- Resize ảnh (255x255)
- Chuẩn hoá pixel
- Chia tập:
 - 85% training (54,324 file ảnh)
 - 7.5% validation (3,891 file ảnh)
 - 7.5% test (4,044 file ảnh)



2.1.3. Xây dựng mô hình CNN bằng Python + TensorFlow/Keras với bài toán phân loại giống cây bệnh

1. Tổng quan bài toán

- Input: ảnh lá cây (ví dụ: healthy, bệnh A, bệnh B...)
- Output: nhãn phân loại (class)

→ Đây là bài toán Image Classification với CNN

2. Chuẩn bị môi trường trên Pycharm

Cài các thư viện cần thiết:

```
pip install tensorflow keras numpy matplotlib opencv-python
```

- TensorFlow: Đây là framework chính để xây dựng và huấn luyện mô hình CNN, tính toán tensor (ma trận nhiều chiều), tối ưu hoá model (sử dụng backpropagation để giảm sai số)
- Keras: là API cấp cao chạy trên TensorFlow, giúp code model một cách đơn giản hơn, tạo các layer như: Conv2D, Dense, MaxPooling. Nói một cách đơn giản: Keras = giao diện thân thiện để điều khiển TensorFlow
- NumPy: Đây là thư viện xử lý số học, dùng để xử lý các vấn đề liên quan tới toán học như: làm việc với mảng, xử lý dữ liệu ảnh dạng ma trận, tính toán nhanh.
- Matplotlib: Thư viện dùng để vẽ biểu đồ: vẽ accuracy/loss, theo dõi model học tốt hay không
- openCV (opencv-python): Thư viện xử lý ảnh, dùng để đọc ảnh từ file, Resize ảnh, xử lý ảnh trước khi đưa vào CNN

3. Xây dựng mô hình CNN

***Import thư viện**

```
#import thư viện
import tensorflow as tf
from tensorflow import keras
from tensorflow.python.layers.normalization import normalization
from tensorflow.keras import layers
from tensorflow.keras import models
```

*Load dữ liệu

```
train_data = tf.keras.preprocessing.image_dataset_from_directory(
    "/content/training",
    image_size=(150,150),
    seed=123,
    shuffle=True, # Xáo trộn ảnh để mô hình học khách quan hơn
    batch_size=64,
)
```

Found 46393 files belonging to 38 classes.

```
val_data = tf.keras.preprocessing.image_dataset_from_directory(
    "/content/validation",
    image_size=(150,150),
    batch_size=64,
    seed=123,
    shuffle=False, # Thường tập validation không cần xáo trộn để dễ theo dõi kết
)
```

... Found 3853 files belonging to 38 classes.

- *tf.keras.preprocessing.image_dataset_from_directory* là một hàm cực kỳ tiện lợi của Keras. Thay vì phải viết code để mở từng tấm ảnh, thay đổi kích thước và dán nhãn, hàm này sẽ làm hết tất cả những việc đó. Để làm việc này, nó sẽ nhìn vào cấu trúc thư mục mà người dùng đưa đường dẫn (*./dataset/training*, *./dataset/validation*...). Nếu thư mục chứa 2 thư mục con là *apple_black_rot* và *apple_healthy*, nó sẽ tự hiểu 2 thư mục đó thuộc về 2 lớp khác nhau

*Chuẩn hoá dữ liệu

```
#chuẩn hóa dữ liệu
normalization_layer = layers.Rescaling(1./255)

train_data = train_data.map(lambda x, y: (normalization_layer(x), y)) #x là ảnh,
val_data = val_data.map(lambda x, y: (normalization_layer(x), y))
```

- *layers* là một lớp trong TensorFlow/Keras.
Normalization_layer = layers.Rescaling(1./255)
 Do đó, câu lệnh trên có nghĩa là tạo một object từ class *layers* của TensorFlow/Keras, dùng để lưu ảnh dataset.

- Một ảnh kỹ thuật số thông thường (RGB) được cấu tạo từ các điểm ảnh (pixels). Mỗi pixels có giá trị nằm trong khoảng 0-255 (tương ứng với độ sáng từ đen đến trắng). Phép tính $1./255$ giúp chuyển đổi giá trị từ [0-255] về đoạn [0,1]. Lý do cho phép chuyển đổi này là vì: Trong toán học của mạng nơ-ron, các trọng số (weights) thường là những số nhỏ. Nếu đầu vào quá lớn (đến 255), nó có thể làm gradient bị bùng nổ hoặc khiến việc hội tụ cực kỳ chậm. Đưa về [0,1] giúp thuật toán tối ưu và hoạt động mượt mà nhất.
- `map()`: Đây là một hàm của TensorFlow Dataset. Nó đi qua từng "cặp" dữ liệu trong tập `train_data` và áp dụng một công thức biến đổi nào đó.

*Xây dựng mô hình CNN

```
#xay dung mo hinh CNN
model = models.Sequential([#các layer xếp tuần tự từ trên xuống dưới
    layers.Conv2D(32,(3,3), activation='relu', input_shape=(150,150,3)),
    layers.MaxPooling2D((2,2)),

    layers.Conv2D(64,(3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),

    layers.Conv2D(128,(3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),

    layers.Flatten(),
    layers.Dense(128,activation='relu'),#dense layer
    layers.Dense(38,activation='softmax') # so class
])

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Mô hình đi theo công thức: (Conv + ReLU + Pooling) * 3 → Flatten → Dense → Output

* Convolutional layer (lớp tích chập):

- Đây là lớp dùng để trích xuất đặc trưng, sử dụng các kernel 3x3 và thực hiện phép tích chập giữa các giá trị pixel với trọng số của bộ lọc để tạo ra các feature map
- Sau khi có feature map thì thực hiện hàm kích hoạt ReLU để loại bỏ các giá trị âm.
- Số filter tăng dần 32 → 64 → 128 giúp mô hình học được các chi tiết từ đơn giản đến phức tạp, các lớp sâu hơn học được đặc trưng phức tạp hơn (hình dạng, đối tượng)

* Pooling layer (Maxpooling2D)

- Lớp này sẽ lấy giá trị lớn nhất trong vùng feature map 2x2, giúp giảm kích thước ảnh xuống $\frac{1}{2}$. Điều này giúp giảm tính toán và tránh tình trạng overfitting.

* Flatten layer

- Chuyển đổi các ma trận feature map sau khi đã đi qua lớp pooling thành 1 vector duy nhất để chuẩn bị đưa vào các lớp học sâu (Dense layers)

**Dense layer*

- Lớp này kết nối mọi đặc trưng đã học để đưa ra các dự đoán logic

**Output (softmax)*

- Với 38 class và hàm softmax để chuyển đổi thành xác suất với output
Ví dụ với output :
[0.01, 0.02, 0.03, ..., 0.9, ...]
→ class có xác suất cao nhất là kết quả.

***Compile model**

```

1
15
#compile model
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

```

Bước này dùng để biên dịch (compile) mô hình CNN trước khi train. Với cấu hình 2 thứ quan trọng

**Optimizer (trình tối ưu hoá)*

- Optimizer là thuật toán chịu trách nhiệm cập nhật các trọng số (weights) của mô hình dựa trên dữ liệu để giảm thiểu sai số.
- ‘adam’ (Adaptive Moment Estimation), là thuật toán kết hợp ưu điểm của 2 thuật toán khác là AdaGrad và RMSProp. Đây là thuật toán xử lý nhiễu rất tốt, tiêu tốn ít bộ nhớ và tự điều chỉnh tốc độ học (learning rate).

**Loss function (hàm mất mát)*

- Đây là hàm cho biết mô hình đang dự đoán sai bao nhiêu. Mục tiêu của việc huấn luyện là làm cho giá trị này càng nhỏ càng tốt.
- Công thức toán học: Về cơ bản, nó tính toán khoảng cách giữa phân phối xác suất dự đoán (y') và nhãn thực tế (y):

$$Loss = - \sum_{i=1}^N y_i \log(y'_i)$$

**Metrics (thước đo)*

- Đây là chỉ số để lập trình viên theo dõi xem mô hình học tốt đến đâu thông qua ‘accuracy’: tỉ lệ phần trăm số ảnh mà mô hình dự đoán đúng.

***Train model**

```
1) 4m ▶ #train model
    history = model.fit(
        train_data,
        validation_data=val_data,
        epochs=10
    )

... Epoch 1/10
725/725 _____ 58s 70ms/step - accuracy: 0.7210 - loss: 0.9728 - v
Epoch 2/10
725/725 _____ 43s 60ms/step - accuracy: 0.8973 - loss: 0.3243 - v
Epoch 3/10
725/725 _____ 44s 60ms/step - accuracy: 0.9377 - loss: 0.1861 - v
Epoch 4/10
725/725 _____ 43s 59ms/step - accuracy: 0.9601 - loss: 0.1176 - v
Epoch 5/10
725/725 _____ 82s 59ms/step - accuracy: 0.9722 - loss: 0.0811 - v
Epoch 6/10
725/725 _____ 83s 61ms/step - accuracy: 0.9793 - loss: 0.0626 - v
Epoch 7/10
725/725 _____ 44s 60ms/step - accuracy: 0.9805 - loss: 0.0586 - v
Epoch 8/10
725/725 _____ 43s 59ms/step - accuracy: 0.9864 - loss: 0.0411 - v
Epoch 9/10
725/725 _____ 81s 58ms/step - accuracy: 0.9834 - loss: 0.0488 - v
Epoch 10/10
725/725 _____ 45s 62ms/step - accuracy: 0.9866 - loss: 0.0384 - v
```

Bước này cho model học từ dữ liệu train và kiểm tra trên dữ liệu validation

- ‘train_data’ gồm input và label, mô hình sẽ học từ input và so sánh với label → sửa sai → lặp lại
- Validation_data sẽ trả về val_loss & val_accuracy: để xem xét xem mô hình có học tốt thật không hay đơn giản chỉ là đang học thuộc (overfitting)
- ‘epochs = 10’ cho mô hình học toàn bộ dataset 10 lần (1 epoch học hết 1 vòng dataset)

*Lưu model

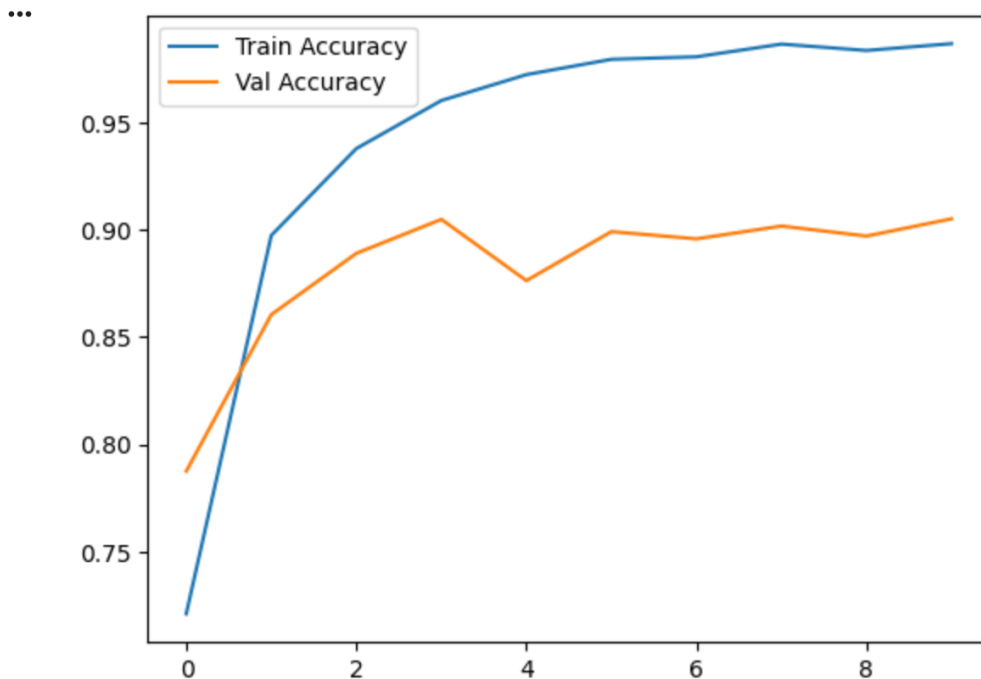
```
▶ # Lưu vào bộ nhớ tạm của Colab
model.save("plant_disease_model.keras")
|
# Copy file đó sang Google Drive cho chắc ăn (đã mount Drive trước đó)
!cp plant_disease_model.keras /content/drive/MyDrive/
```

Lưu model thành file ‘plant_disease_model.keras’.

2.2. Đánh giá

2.2.1. Accuracy

```
▶ plt.plot(history.history['accuracy'], label='Train Accuracy')  
plt.plot(history.history['val_accuracy'], label='Val Accuracy')  
plt.legend()  
plt.show()
```



Phân tích giá trị Accuracy

- Training Accuracy tăng nhanh qua các epoch và đạt giá trị rất cao, xấp xỉ 98 - 99%. Điều này cho thấy mô hình đã học rất tốt các đặc trưng từ tập dữ liệu huấn luyện.
- Validation Accuracy đạt khoảng 90% và gần như không cải thiện thêm kể từ khoảng epoch thứ 3, có xu hướng đi ngang trong các epoch tiếp theo.

Hiện tượng Overfitting

Từ biểu đồ trên, có thể nhận thấy mô hình bắt đầu xuất hiện dấu hiệu

Overfitting(quá khớp):

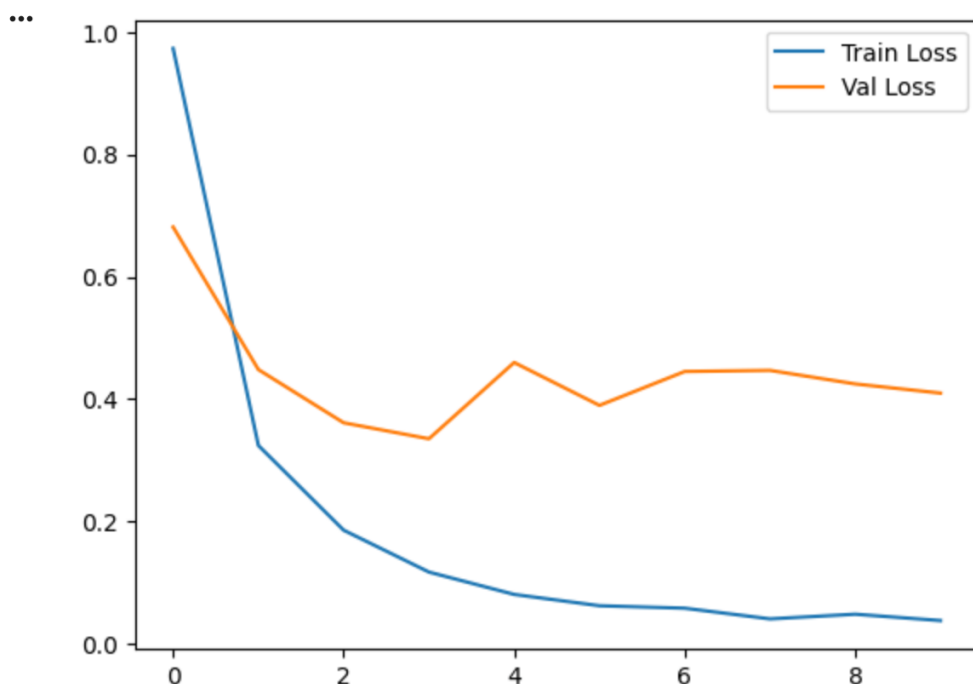
- Khoảng cách giữa Training và Validation Accuracy vào khoảng 8 - 9%, vượt ngưỡng 5% - đây là dấu hiệu cho thấy mô hình đang học quá chi tiết dữ liệu huấn luyện và giảm khả năng tổng quát hóa.
- Hiện tượng bão hòa (saturation): Sau epoch thứ 3, Validation Accuracy không tăng thêm, trong khi Training Accuracy vẫn tiếp tục cải thiện. Điều này cho thấy việc tiếp tục huấn luyện không mang lại hiệu quả thực tế mà chỉ làm mô hình phù hợp hơn với dữ liệu huấn luyện.

Đánh giá hiệu năng mô hình

- Với bài toán phân loại bệnh trên cây trồng, vốn có nhiều lớp dữ liệu tương đồng và khó phân biệt, việc đạt 90% Validation Accuracy được xem là một kết quả tốt và chấp nhận được trong phạm vi đề án.
- Mặc dù có dấu hiệu overfitting nhẹ, đường Validation Accuracy vẫn ổn định và không suy giảm mạnh, cho thấy mô hình vẫn giữ được khả năng dự đoán tương đối tốt trên dữ liệu chưa thấy.

2.2.2. Loss

```
import matplotlib.pyplot as plt
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.legend()
plt.show()
```



Dựa trên biểu đồ Loss của tập huấn luyện (Training) và tập kiểm định (Validation), có thể đưa ra các nhận xét sau:

Quan sát xu hướng Loss

- Training Loss giảm nhanh và tiến gần về giá trị 0.0, cho thấy mô hình đã học rất tốt và tối ưu hóa hiệu quả trên tập dữ liệu huấn luyện.
- Validation Loss giảm trong những epoch đầu (khoảng epoch 1–3), sau đó có xu hướng đi ngang và dao động nhẹ, thậm chí tăng ở một số thời điểm.

Hiện tượng Overfitting

Sự khác biệt giữa hai đường Loss cho thấy mô hình xuất hiện hiện tượng Overfitting

- Khi Training Loss tiếp tục giảm nhưng Validation Loss không giảm tương ứng, điều này chứng tỏ mô hình đang dần ghi nhớ dữ liệu huấn luyện thay vì học các đặc trưng tổng quát.

- Khoảng cách giữa hai đường Loss ngày càng tăng, phản ánh mức độ quá khớp của mô hình.

Tốc độ hội tụ và điểm dừng tối ưu

- Mô hình hội tụ nhanh, đạt giá trị Loss thấp chỉ sau khoảng 2 - 3 epoch đầu.
- Epoch tối ưu có thể xác định vào khoảng epoch thứ 3, tại đó Validation Loss đạt giá trị thấp nhất.
- Sau mốc này, việc tiếp tục huấn luyện không giúp cải thiện hiệu năng trên dữ liệu mới mà còn có thể làm giảm khả năng tổng quát hóa.

Đánh giá hiệu năng

- Mặc dù xuất hiện overfitting, Validation Loss vẫn duy trì ở mức tương đối thấp (~ 0.4), cho thấy mô hình vẫn có khả năng dự đoán tốt trên dữ liệu chưa thấy.
- Do đó, mô hình đạt mức hiệu năng chấp nhận được trong bối cảnh bài toán phân loại ảnh giống cây bệnh.

2.2.3. So sánh với model có sẵn (MobileNet)

*Load model MobileNetV2

```
[20]
✓ Os from tensorflow.keras.applications import MobileNetV2

base_model = MobileNetV2(weights='imagenet', include_top=False)
```

*Train model MobileNetV2 trên dataset của bài toán phân loại giống cây bệnh

```
history2 = model.fit(train_data, validation_data=val_data, epochs=10)
```

```
Epoch 1/10
725/725 ————— 42s 58ms/step - accuracy: 0.9942 - loss: 0.0176 - v
Epoch 2/10
725/725 ————— 86s 64ms/step - accuracy: 0.9937 - loss: 0.0197 - v
Epoch 3/10
725/725 ————— 44s 60ms/step - accuracy: 0.9925 - loss: 0.0239 - v
Epoch 4/10
725/725 ————— 44s 61ms/step - accuracy: 0.9957 - loss: 0.0145 - v
Epoch 5/10
725/725 ————— 44s 61ms/step - accuracy: 0.9934 - loss: 0.0221 - v
Epoch 6/10
725/725 ————— 82s 60ms/step - accuracy: 0.9960 - loss: 0.0157 - v
Epoch 7/10
725/725 ————— 41s 57ms/step - accuracy: 0.9954 - loss: 0.0158 - v
Epoch 8/10
725/725 ————— 82s 57ms/step - accuracy: 0.9934 - loss: 0.0243 - v
Epoch 9/10
725/725 ————— 43s 58ms/step - accuracy: 0.9959 - loss: 0.0139 - v
Epoch 10/10
725/725 ————— 44s 60ms/step - accuracy: 0.9947 - loss: 0.0168 - v
```

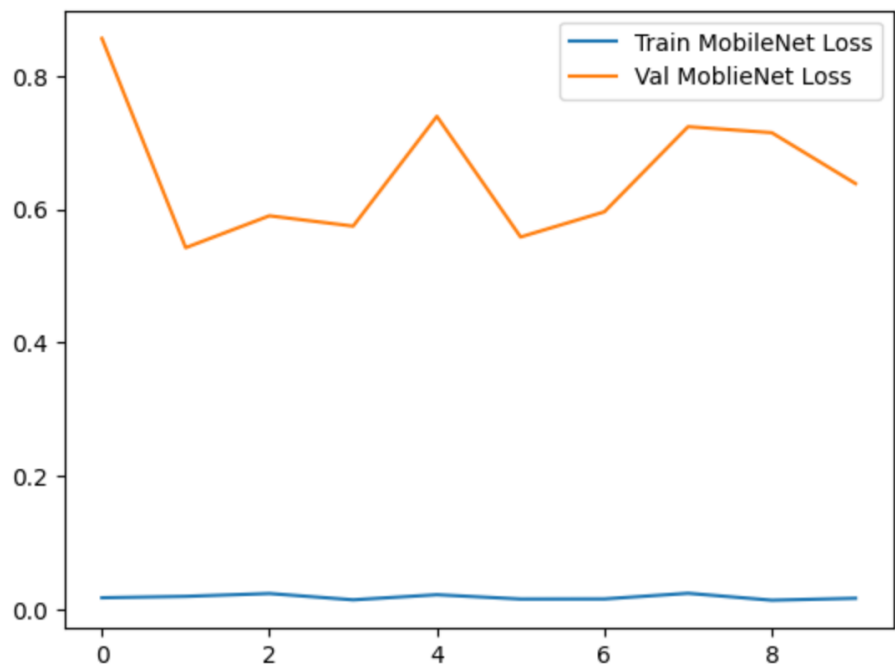
*Lưu model thành file 'plant_disease_model_mobilenet.keras'

```
# Lưu vào bộ nhớ tạm của Colab
model.save("plant_disease_model_mobilenet.keras")

# Copy file đó sang Google Drive cho chắc ăn (đã mount Drive trước đó)
!cp plant_disease_model_mobilenet.keras /content/drive/MyDrive/
```

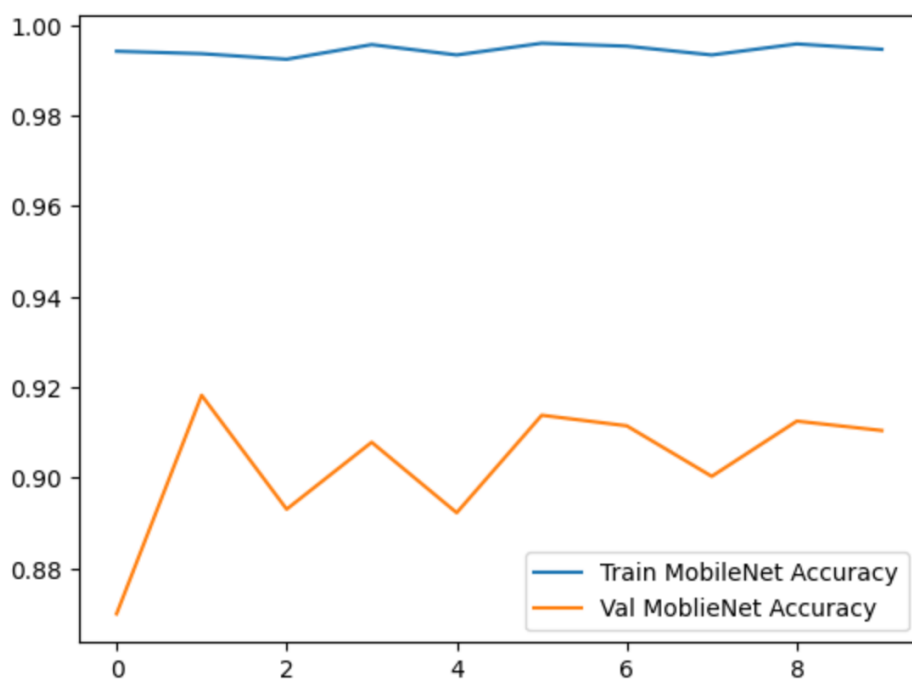
*Biểu đồ loss của model MobileNet

```
plt.plot(history2.history['loss'], label='Train MobileNet Loss')
plt.plot(history2.history['val_loss'], label='Val MoblieNet Loss')
plt.legend()
plt.show()
```



**Biểu đồ Accuracy của model MobileNet*

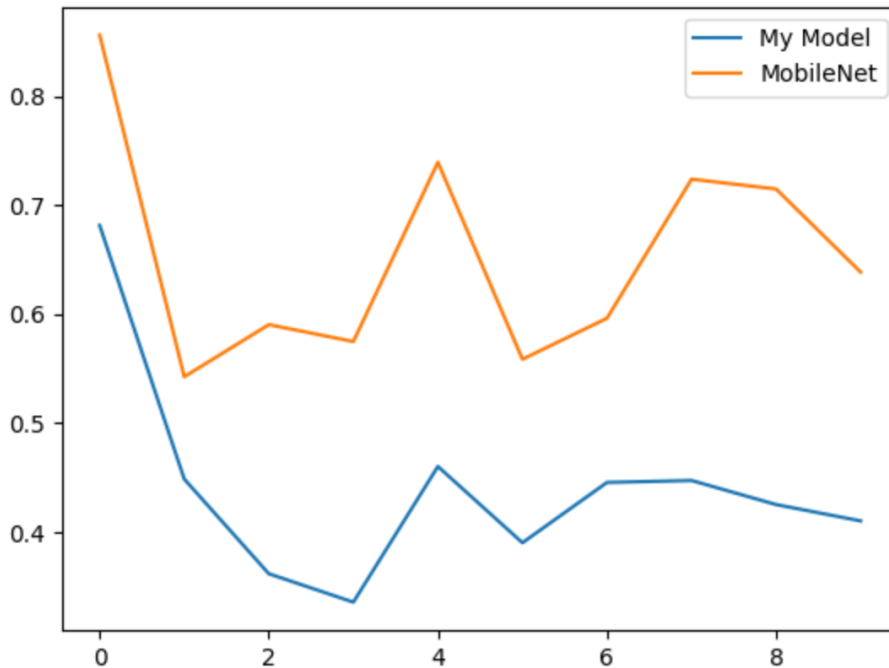
```
plt.plot(history2.history['accuracy'], label='Train MobileNet Accuracy')
plt.plot(history2.history['val_accuracy'], label='Val MoblieNet Accuracy')
plt.legend()
plt.show()
```



**So sánh kết quả loss function giữa model đã tạo và MobileNet Model*

```
# so sánh loss
plt.plot(history.history['val_loss'], label='My Model')
plt.plot(history2.history['val_loss'], label='MobileNet')

plt.legend()
plt.show()
```



Biểu đồ Validation Loss thể hiện rõ sự khác biệt về hiệu năng giữa hai mô hình, qua đó cho phép đánh giá trực tiếp khả năng tổng quát hóa trên dữ liệu chưa thấy.

So sánh giá trị Loss

- Mô hình tự tạo (My Model) duy trì mức Validation Loss thấp hơn đáng kể (xấp xỉ 0.4).
- Trong khi đó, mô hình MobileNet có Validation Loss cao hơn, dao động trong khoảng 0.55 - 0.85.

Theo nguyên lý tối ưu hóa, giá trị Loss càng thấp thì sai số dự đoán càng nhỏ, do đó mô hình đề xuất cho thấy hiệu năng tốt hơn trên tập validation.

Độ ổn định và khả năng hội tụ

- Mô hình tự tạo:
 - Giảm nhanh trong các epoch đầu (đặc biệt từ epoch 1 đến 3)
 - Sau đó duy trì ổn định với biên độ dao động nhỏ.
→ Cho thấy quá trình hội tụ tốt và ổn định
- Mô hình MobileNet:
 - Dao động mạnh giữa các epoch
 - Không có xu hướng hội tụ rõ ràng
→ Thể hiện sự thiếu ổn định trong quá trình tối ưu trên dữ liệu validation

Đánh giá tổng quát

Kết quả thực nghiệm cho thấy mô hình tự tạo vượt trội hơn so với MobileNet trong bối cảnh bài toán phân loại ảnh giống cây bệnh:

- Khả năng tổng quát hóa: Tốt hơn (do Validation Loss thấp hơn)
- Độ ổn định: Cao hơn (dao động ít hơn)
- Mức độ phù hợp: Phù hợp hơn với đặc thù dữ liệu ảnh bệnh cây trồng

Giải thích nguyên nhân

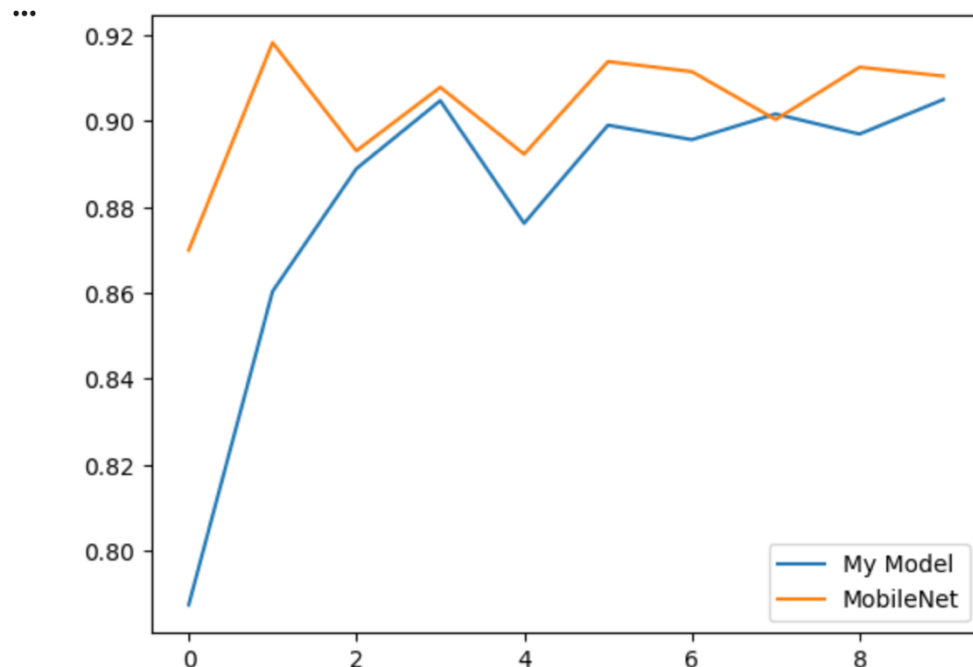
Một số nguyên nhân có thể dẫn đến kết quả trên:

- **Tính chuyên biệt của mô hình:**
Mô hình đề xuất có cấu trúc đơn giản và được thiết kế phù hợp với bài toán, giúp học các đặc trưng quan trọng của dữ liệu một cách hiệu quả.
- **Độ phức tạp của MobileNet:**
MobileNet là mô hình sâu và được huấn luyện trên tập dữ liệu lớn như ImageNet, do đó có thể không phù hợp hoàn toàn với bài toán phân loại bệnh lá cây, dẫn đến hiện tượng overfitting.
- **Sự khác biệt miền dữ liệu:**
Các đặc trưng học từ dữ liệu tổng quát (động vật, phương tiện,...) có thể không tối ưu cho việc nhận diện các chi tiết nhỏ như đốm bệnh trên lá.
→ Dựa trên biểu đồ Validation Loss, mô hình đề xuất cho thấy hiệu năng vượt trội và ổn định hơn so với MobileNet. Điều này chứng minh rằng việc lựa chọn mô hình phù hợp với bài toán và dữ liệu cụ thể có ý nghĩa quan trọng hơn so với việc sử dụng các mô hình phức tạp có sẵn.

*So sánh kết quả accuracy giữa model đã tạo và MobileNet model

```
# so sánh accuracy
plt.plot(history.history['val_accuracy'], label='My Model')
plt.plot(history2.history['val_accuracy'], label='MobileNet')

plt.legend()
plt.show()
```



Biểu đồ Validation Accuracy cung cấp một góc nhìn bổ sung so với biểu đồ Loss, cho thấy sự khác biệt đáng chú ý giữa hai mô hình.

Hiện tượng khác biệt giữa Accuracy và Loss

- Mô hình MobileNet đạt Validation Accuracy cao hơn (trên 91%).
 - Trong khi đó, mô hình tự tạo đạt khoảng 90% - 90.5%, thấp hơn một chút.
- Tuy nhiên, kết quả này không hoàn toàn đồng nhất với biểu đồ Loss, khi MobileNet lại có Validation Loss cao và dao động mạnh hơn.

Giải thích:

Accuracy chỉ phản ánh tỷ lệ dự đoán đúng/sai, trong khi Loss đo lường mức độ sai lệch xác suất. Do đó:

- MobileNet có thể dự đoán đúng với xác suất rất cao → giúp tăng Accuracy
- Nhưng khi dự đoán sai, sai số rất lớn (xác suất lệch xa giá trị thực) → làm Loss tăng mạnh

Ngược lại, mô hình đề xuất có xu hướng dự đoán “thận trọng” hơn, nên tổng sai số (Loss) thấp hơn dù Accuracy không cao nhất.

Độ ổn định và xu hướng hội tụ

- **Mô hình tự tạo:**
 - Đường Accuracy tăng đều và ổn định qua các epoch

- Thể hiện xu hướng tiếp tục cải thiện nếu được huấn luyện thêm
→ Cho thấy khả năng hội tụ tốt và bền vững
- **Mô hình MobileNet:**
 - Dao động mạnh giữa các epoch (zig-zag)
 - Không duy trì được mức Accuracy ổn định
→ Thể hiện sự thiếu ổn định trong quá trình học

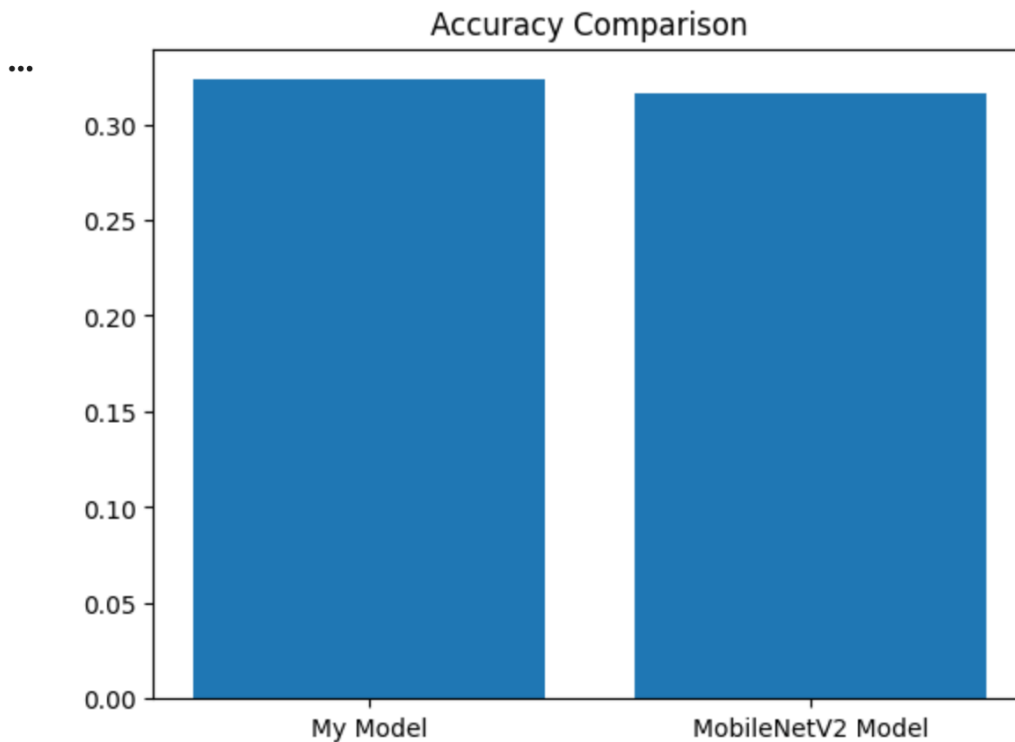
Đánh giá tổng thể

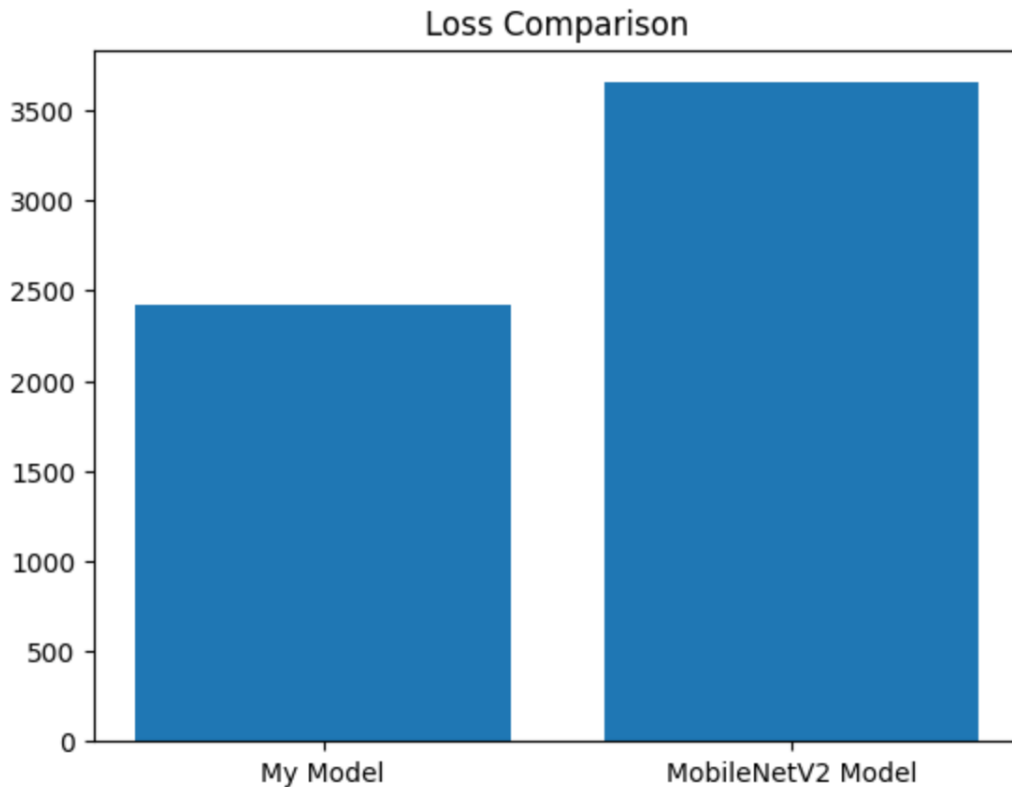
Dựa trên các phân tích đã chỉ ra cho thấy mô hình đề xuất có độ tin cậy cao hơn khi triển khai thực tế, do ít bị ảnh hưởng bởi các dự đoán sai nghiêm trọng.

**So sánh accuracy, loss của 2 model trên tập Test*

```
print("My Model - Loss:", loss1, "- Accuracy:", acc1)
print("MobileNetV2 Model - Loss:", loss2, "- Accuracy:", acc2)
```

```
My Model - Loss: 2420.393310546875 - Accuracy: 0.32326510548591614
MobileNetV2 Model - Loss: 3652.155029296875 - Accuracy: 0.31577634811401367
```





So sánh hiệu năng trên tập Test

Biểu đồ trên thể hiện kết quả đánh giá hai mô hình trên tập dữ liệu Test, giúp phản ánh chính xác khả năng tổng quát hóa trong thực tế.

So sánh Accuracy

- Mô hình đề xuất (My Model) đạt Accuracy khoảng 0.32 - 0.33
- Mô hình MobileNetV2 đạt thấp hơn một chút, khoảng 0.31 - 0.32

→ Sự chênh lệch không quá lớn, tuy nhiên mô hình đề xuất vẫn nhỉnh hơn về độ chính xác trên dữ liệu hoàn toàn mới.

So sánh Loss

- My Model có Loss khoảng ~2400
- MobileNetV2 có Loss cao hơn đáng kể, khoảng ~3600 - 3700

→ Điều này cho thấy:

- Mô hình đề xuất có sai số tổng thể thấp hơn
- MobileNetV2 có nhiều dự đoán sai với mức độ sai lệch lớn

Đánh giá tổng thể trên Test set

Kết quả trên tập Test củng cố lại các nhận định trước đó:

- Model tự tạo:
 - Accuracy cao hơn
 - Loss thấp hơn
 - → Khả năng tổng quát hóa tốt hơn và đáng tin cậy hơn
- MobileNetV2:
 - Accuracy thấp hơn nhẹ
 - Loss cao hơn rõ rệt
 - → Hiệu năng kém ổn định, có dấu hiệu dự đoán sai nghiêm trọng

Mặc dù MobileNetV2 là mô hình mạnh hơn về lý thuyết, nhưng trong bài toán cụ thể này, mô hình tự xây dựng lại cho kết quả tốt hơn trên tập Test.

Điều này chứng minh rằng:

- Mô hình phù hợp với dữ liệu quan trọng hơn mô hình phức tạp
- Transfer Learning không phải lúc nào cũng vượt trội nếu không được tinh chỉnh đúng cách

Kết luận

Dựa trên cả hai tiêu chí Accuracy và Loss trên tập Test:

Mô hình tự tạo (My Model) cho thấy hiệu năng vượt trội hơn so với MobileNetV2, với độ chính xác cao hơn và sai số thấp hơn.

III. Tài liệu tham khảo

1. <https://viblo.asia/p/gioi-thieu-ve-deep-learning-deep-learning-hoat-dong-nhu-the-nao-3Q75w2nDIWb>
2. <https://vnptai.io/vi/blog/detail/neural-network-la-gi>
3. <https://vnptai.io/vi/blog/detail/convolutional-neural-networks-la-gi>
4. <https://www.youtube.com/watch?v=UIPmxeyRWdA>